# 0050. Pow(x, n)

Medium | Category: Math & Geometry | LeetCode: https://leetcode.com/problems

# Problem

- Compute x raised to the integer power n (n may be negative).

# Intuition

- Fast exponentiation: x^n = (x^2)^(n/2), times x if n is odd. Halving n each step gives O(log n).
- Negative n: compute (1/x)^(-n).

# Brute Force

- Multiply n times: O(n), too slow for n near 2^31.

# Optimal Approach

- E = (long) n; if e < 0: x = 1/x, e = -e.
- Res = 1. While e > 0: if e is odd, res = x; x = x; e >>= 1.

# Worked Example

```
x = 2, n = 10 (binary 1010).

e=10 even           x=4
e=5  odd  res=4     x=16
e=2  even           x=256
e=1  odd  res=1024
```

# Java Solution

```java
public class PowXN {
    public double myPow(double x, int n) {
        long e = n;
        if (e < 0) { x = 1 / x; e = -e; }
        double res = 1;
        while (e > 0) {
            if ((e & 1) == 1) res *= x;
            x *= x;
            e >>= 1;
        }
        return res;
    }
}
```

# Complexity

- Time: O(log n). Space: O(1).

# Pitfalls

- -Integer.MIN_VALUE overflows int; convert to long first.